Manil Bastola
Stephanie Mertz
CSE 566S
Homework 4

All experiments in this paper were performed on Wustl Linux Lab server. Experiments pertaining to Problem 1 was conducted on single desktop session running on vnc.q queue while other experiments were conducted on multiple servers all running on all.q queue cluster.

1.) What is the performance dependence upon the size of the problem? There are several parameters that could be considered "size of the problem" here, including many that are in the code but not exposed on the command line, so multiple investigations will be warranted

To analyze the effect of size of the problem on performance, we looked at three aspects to problem size - the number of stars (N), the time step (Δt), and the mass of the stars (M), changing one of each while keeping others constant. N was an obvious candidate for problem size. We expected Δt to affect the program performance in terms of programs numeric precision (low Δt means high precision). M was tested for curiosity to check whether or not the program considered each star object a point particle, a standard technique in physics for complexity reduction.

Since the N-body problem in this case pertains to N bodies (stars in galaxies) interacting with some “force” on one another, we expect the runtime to be in the range of O(N^2). This is because each force can be thought of as an edge between two vertices (stars) with total of N choose 2 edges. Observing the code, it seems that t-step only affected the integration rules (updateEuler, updateRKutta4,etc). Hence we hypothesize that the performance dependencies on the time step to be linear but inverse $O(1/\Delta t)$. M comes to play only when calculating the energy in modeldata.cpp so the dependence with respect to M is expected to be O(1). Other parameters are not considered since they depended on N, M or Δt. For example, the method of integration and t_final depends on Δt.

While investigating the role of varying N, we kept the M = 200.0, t-final = 1000 and $\Delta t = 8.0$ and performed 5 repetitions for statistical significance. We ran the experiment on Linux Lab Desktop with number of processes = 4. The error bars were calculated using student t-tests. We observed a P = N^2 relationship where N is number of stars and P is performance. Trials had low variability so the error bars below are difficult to notice.

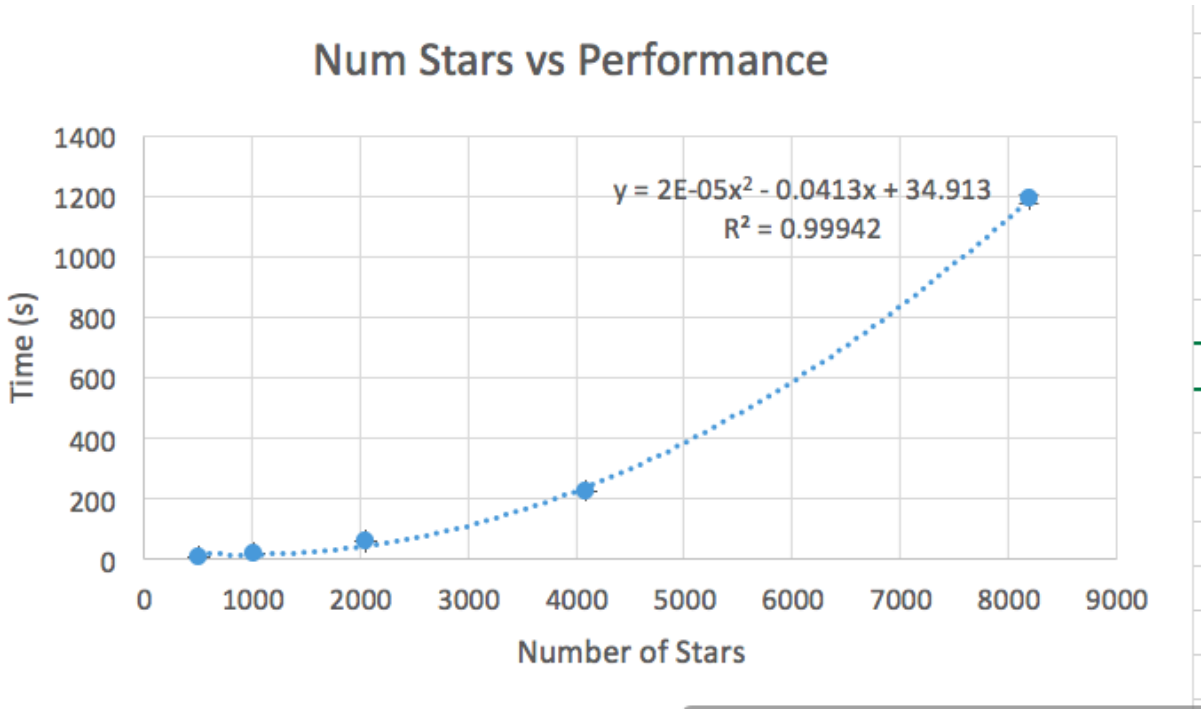


fig 1: Running time vs N. M = 200.0, t-final = 1000, Δt = 8.0 and mpi numprocesses = 4.

While investigating the role of varying M, we kept the N = 1024, t-final = 1000 and Δt = 8.0 and performed 5 repetitions for statistical significance. We ran the experiment on Linux Lab Desktop with number of processes = 4. As expected, we did not observe any relationship between mass of the stars and the performance of the program.

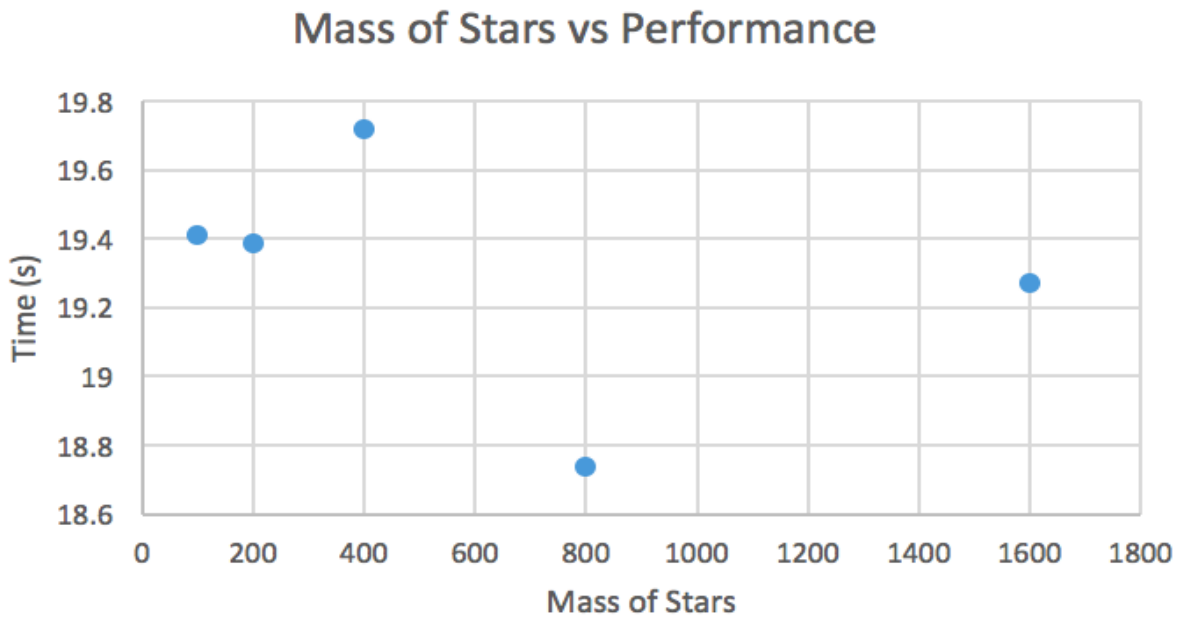


fig 2: Running time vs Δt. N = 1024, t-final = 1000, M = 200.0 and mpi numprocesses = 4.

While investigating the role of varying Δt , we kept the N = 1024, M = 2000, t-final = 1000 and performed 5 repetitions for statistical significance. We ran the experiment on Linux Lab Desktop with number of processes = 4. As expected we observed inverse relationship of performance vs Δt as shown below.

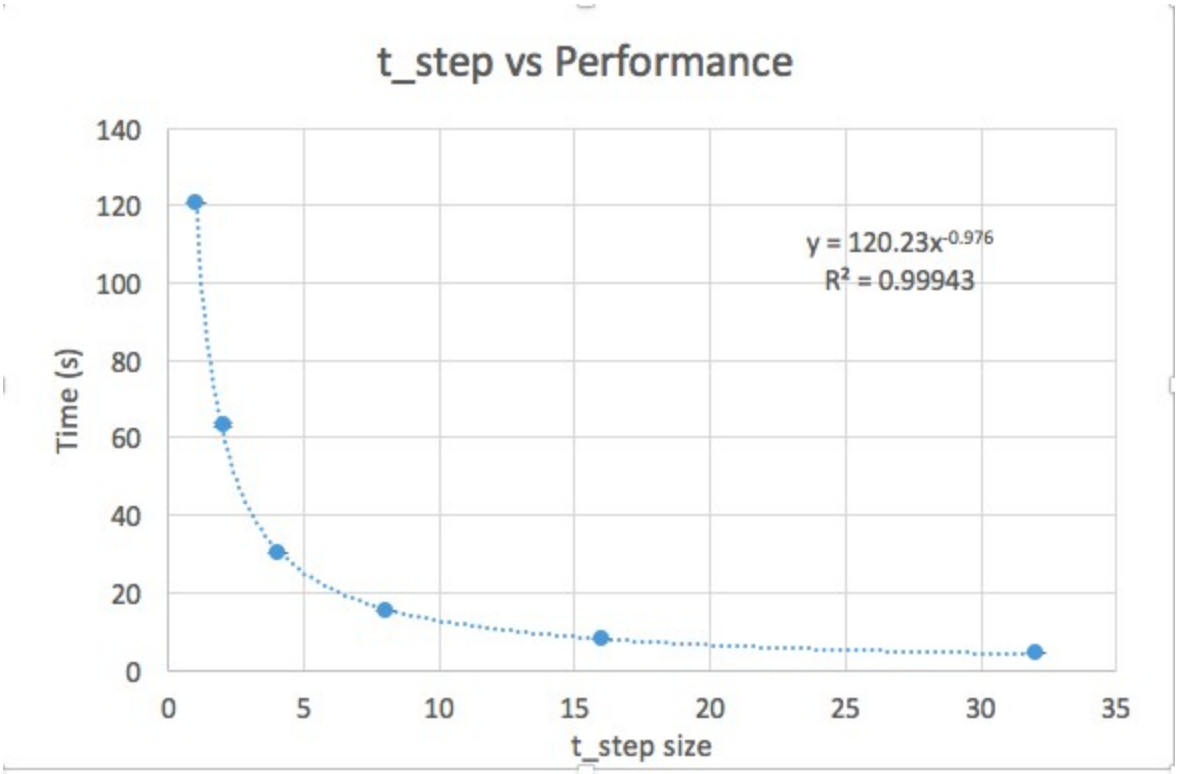


fig 3: Running time vs M. N = 1024, t-final = 1000 and Δt = 8.0 and mpi numprocesses = 4.

2.) How does the program perform as the number of nodes (processes) is varied? Especially pay attention to what happens when you cross system boundaries (e.g., every 4 nodes).

We do not expect the N-body simulation algorithm to be embarrassingly parallel. This is because although force calculation can be divided over each core for each pair, some information about each star object has to be shared within the cores. Hence, there exists overhead due to communication.

To investigate the role of varying num processes, we kept the N = 1024, t-final = 1000 and Δt = 8.0 and performed 5 repetitions for statistical significance. We ran the experiment on Linux Lab server. We found that the performance increased as nodes were added until the cost of communication overtook the gain in performance for having more processors. We noticed this point to be at 16 nodes. We observed that the system boundary for each server was 8 nodes i.e. mpi tasks seeking more than 8 nodes were divided into 2 servers, more than 16 into 3 servers, and so on. Num processes = 16 had the best performance implies that the

communication cost between two servers was not comparable to the benefits gained in computation. For more than two servers, the cost of communication overrides the additional processing power showing weak parallelization ability with the naive MPI implementation.

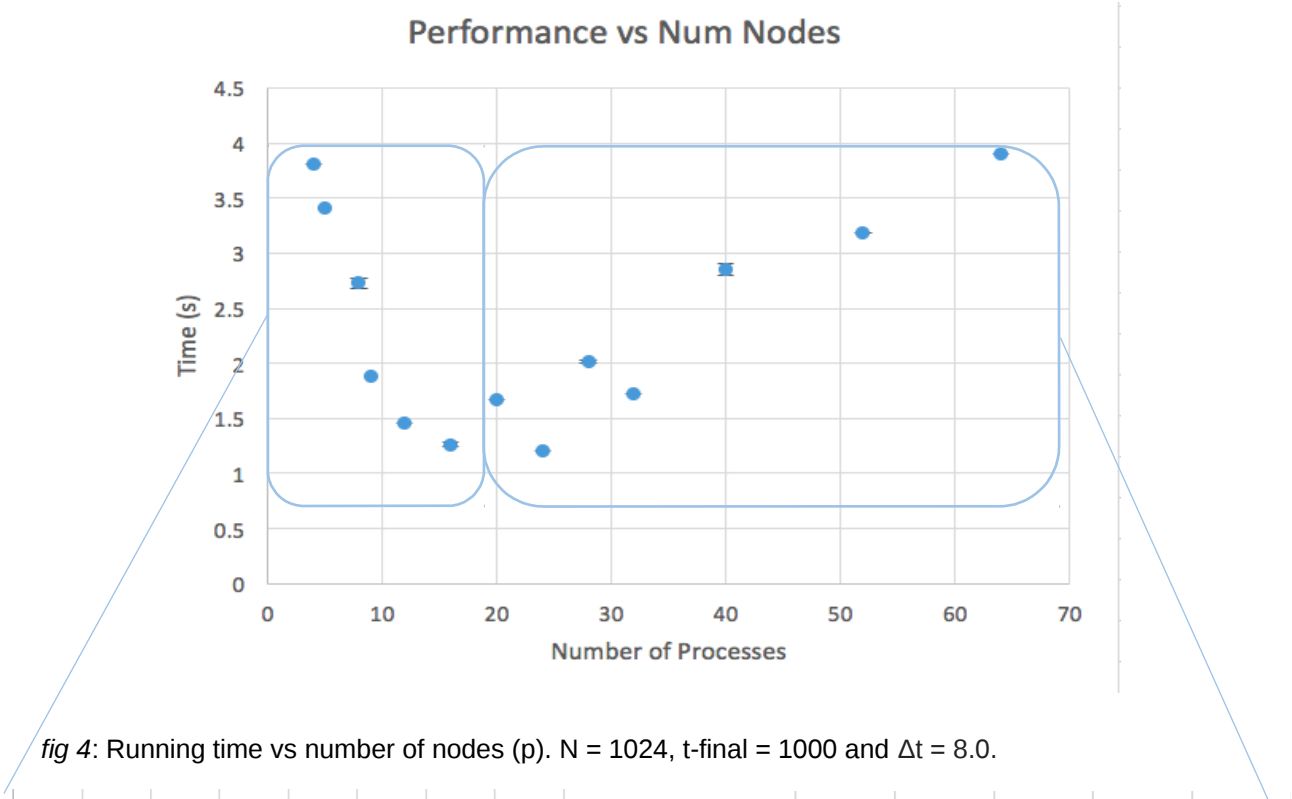


fig 4: Running time vs number of nodes (p). N = 1024, t-final = 1000 and Δt = 8.0.

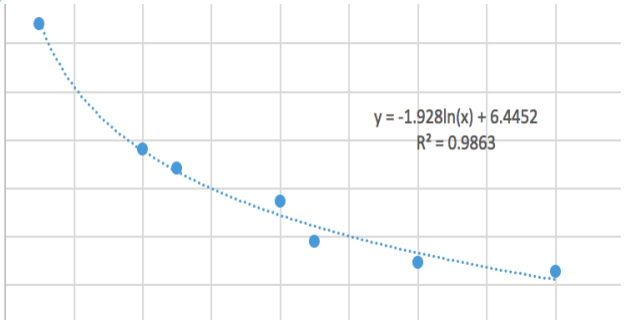


fig 4a: Figure 4, nodes <= 16

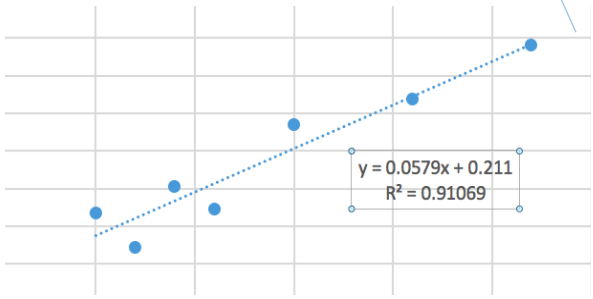


fig 4b: Figure 4, nodes > 16

The relationship between number of nodes and performance is quite inversely linear up until 16. The code is able to take advantage of additional processing power while communicating within shared memory. As the number of nodes surpasses 16, the relationship becomes directly linear as more processors increase, the amount of communication causes the performance to increase linearly.

3.) Based on what you've learned, improve the performance of this program. I am not looking for the best possible execution, any performance improvement will do. There are many potential improvements to be made!

Although there are clever algorithms to significantly improve N-body simulations (O(N*log*N) with P3M[1]), we focused on local optimizations on the default program. We tried three variations: Firstly we tried to OpenMP certain parallelizable sections in diffeq.cpp. For example, in function updateEuler:

```
#pragma omp parallel for
for(int i=0;i<neq;i++){
        x[i]=x[i]+step*der[i];
}
```

We used omp pragmas for all for loops in the diffeq file. The resulting performance difference was insignificant for both large and small number of processes in MPI. We argue that this is due to saturation of parallelization abilities using MPI. We think that limiting the number of MPI process per cpu on each cluster computer combined with OpenMP shared for loop can improve the performance due to shared memory access ability of OpenMP while still benefitting from the power of distributed computing with MPI (superior performance of OpenMP to MPI with regard to communication was observed in hw3). However, we were unsuccessful with assigning certain number of processes per cpu on each cluster computer using the MPI.

In Monte Carlo implemented in hw2, we noticed significant performance gains from paralyzing random numbers using rand_r(). Hence, we replaced instances we found in modeldata.cpp where rand() function was used in each star generation. The gain the performance however was negligible. For example,for 16 processes, the running time of the program with N = 1024, t-final = 1000 and $\Delta t = 8.0$ was 1.167± 0.03s with rand() while 1.087± 0.032s using rand_r(). implementation. Similarly for N = 2048 was, 4.226 ± 0.15s with rand() while 4.173± 0.09s using rand_r() implementation.

Finally we resorted to compiler based optimizations. Using -O3 flag while building the program significantly increased (4x) the performance. For example, with 1 process, the running time of the program with N = 1024, t-final = 1000 and $\Delta t = 8.0$ was 6.46± 0.1s while with -O3 optimization, the running time was 1.601± 0.05s. With num process=16 and all else kept constant, the running time of the program was 1.167 ± 0.03s while the optimized program ran in 0.342 ± 0.25s.

4.) Put together a performance model of your best-executing code that attempts to include as many of the above factors as is reasonable.

From questions 1) and 2) we deduce that the performance of the program should take the form of performance $\tau = a N^2/p\,\Delta t + bNp$ where a, b are constants while p is the number of

1 http://www.shodor.org/media/content//jocse/volume1/issue1/jocse_vol1_issue1

parallel processes spawned by the program. The term $a N^2 / p \Delta t$ corresponds to the computing portion of the program. This was shown to be O(N^2) and O(1 / Δt) in problem 1 while $\tau \sim 1/p$ was shown in question 2. This effect of p is significant within the range $0<p<16$. The term bNp pertains to the communication cost of sharing some information (acceleration, velocity, position, etc) of each star object at each p processors. This term becomes significant in the range $p>16$ as shown in question 2. The tension between the two terms is shown by the logarithmic decrease (saturation in linear decrease) of running time in the range $0<p<16$ (fig 4a) and a linear increase after $p>16$ (fig 4b).

References:

1) Nocito, Nicholas W. “Parallelization of Particle-Particle, Particle-Mesh Method within N-Body Simulation”. *Journal of Computer Science Education*, Dec 2010, Vol1 Issue 1
2) Wikipedia contributors. "N-body simulation." *Wikipedia, The Free Encyclopedia*. Wikipedia, The Free Encyclopedia, 28 Feb. 2017. Web. 27 Mar. 2017.
3) Wikipedia contributors. "Runge–Kutta methods." *Wikipedia, The Free Encyclopedia*. Wikipedia, The Free Encyclopedia, 21 Mar. 2017. Web. 27 Mar. 2017.